"SISTEMA DE GESTIÓN DE INVENTARIO JORGE MOTORS"

Plataforma web basada en arquitectura en capas, patrones de diseño GoF y modelo C4

AUTOR(ES):
CASTILLO CANALES, KORAYMA ANAÍS
HERNÁNDEZ HERNÁNDEZ, JOSÉ ANTONIO
NEYRA QUIROZ, JOSÉ GUILLERMO
ROJAS CASTILLO, SEBASTIÁN JAREN

DOCENTE:
MG. LUIS ENRIQUE RAMÍREZ PACHECO

CURSO:
ARQUITECTURA DE SOFTWARE – VI – MAÑANA – A1

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

1. Introducción

Jorge Motors es una mecánica automotriz de Pueblo Nuevo, Chincha (Ica), que gestiona repuestos, lubricantes, filtros, baterías, herramientas y accesorios. Nuestro proyecto diseña una **plataforma web** que automatiza y centraliza ese inventario: una SPA en React, una API REST en Node.js/Express con TypeScript y una base de datos relacional con Prisma ORM, organizadas en tres capas.

Problema actual: Excel

- Registros con errores o desactualizados.
- El stock no se actualiza solo tras cada entrada o salida.
- Sin alertas cuando un producto llega a nivel crítico.

¿Por qué importa la arquitectura?

Recoge las decisiones más costosas de cambiar y define la calidad del sistema: modularidad, escalabilidad y mantenibilidad (Bass et al., 2021).

Arquitectura en capas

Presentación, negocio y datos separados: cada capa cumple una función y un cambio queda acotado a un módulo.

Patrones de diseño GoF

Seis patrones resuelven necesidades concretas: Singleton, Facade, Observer, Strategy, State y Chain of Responsibility.

Modelado C4

Cuatro niveles de zoom (contexto, contenedores, componentes y código) para explicar el sistema al dueño y al equipo.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

2. Objetivos

Objetivo general

Diseñar y modelar la arquitectura de software de una plataforma web segura para la gestión de inventario de “Jorge Motors”, aplicando arquitectura en capas, patrones de diseño y el modelo C4, de modo que se automatice el control de entradas y salidas, se reduzcan los errores de registro y se incorporen alertas de reabastecimiento.

Objetivos específicos

1. Analizar requerimientos
Identificar los requerimientos funcionales y no funcionales a partir del problema del taller.

2. Seleccionar patrones
Identificar y justificar los patrones arquitectónicos y de diseño (GoF) adecuados al contexto.

3. Modelar con C4
Representar la arquitectura en contexto, contenedores, componentes y código.

4. Documentar decisiones
Registrar la solución técnica, sus atributos de calidad y la justificación de cada decisión.

5. Diseñar datos e interfaz
Elaborar el modelo de datos y los prototipos que servirán de base para la implementación.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

3. Descripción del proyecto web

Contexto y problemática

Jorge Motors registra hoy su inventario en hojas de Excel, lo que dificulta el seguimiento de existencias y puede causar desabastecimiento.

La plataforma permite:

- Controlar entradas y salidas de repuestos en tiempo real.
- Diferenciar permisos de los usuarios según su rol.
- Avisar automáticamente al proveedor cuando un producto cae bajo su mínimo.
- Apoyar la toma de decisiones con reportes de inventario y kardex.

Usuarios del sistema

Administrador del taller
Usuario primario. Gestiona usuarios, productos, proveedores y reportes; supervisa el stock.

Mecánico / operador
Usuario primario. Registra las salidas de repuestos y consulta el stock disponible.

Proveedor de repuestos
Actor externo. Recibe las alertas por correo, SMS o WhatsApp; no usa la interfaz.

Cliente del taller
Beneficiario indirecto: hay stock disponible y la atención es más rápida.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

3. Descripción del proyecto web: funcionalidades

Autenticación y roles
Inicio de sesión con JWT; permisos de ADMINISTRADOR o MECÁNICO. (RF-01 a RF-04)

Productos y categorías
Catálogo de repuestos e insumos clasificado por categorías. (RF-05, RF-06)

Proveedores
Registro de proveedores y de su canal de aviso preferido. (RF-07)

Inventario
Entradas y salidas con actualización inmediata del stock. (RF-08, RF-09)

Movimientos
Historial inmutable con usuario, fecha y stock antes y después. (RF-10)

Reportes
Inventario valorizado y kardex por producto. (RF-11)

Alertas de bajo stock
Aviso automático al proveedor cuando el stock cae bajo el mínimo. (RF-12)

12 RF · 14 RNF
requerimientos derivados del contexto del taller

Alcance

Dentro del alcance
Catálogo, inventario en tiempo real, alertas a proveedores, usuarios por rol y reportes.

Fuera del alcance
Contabilidad general, diagnóstico de vehículos, órdenes de trabajo y facturación a clientes.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

3. Descripción del proyecto web: requerimientos funcionales

ID	Requerimiento	Módulo	Descripción
RF-01	Inicio de sesión	Autenticación	Iniciar sesión con credenciales válidas y recibir un token JWT.
RF-02	Diferenciación de roles	Autorización	Aplicar permisos según el rol: ADMINISTRADOR o MECÁNICO.
RF-03	Restricción por rol	Autorización	Restringir las acciones de la API según el rol autenticado.
RF-04	Usuario autenticado	Autenticación	Consultar la información del usuario que inició sesión.
RF-05	Gestión de productos	Productos	Registrar, consultar, actualizar y administrar productos.
RF-06	Gestión de categorías	Productos	Administrar las categorías que clasifican los productos.

ID	Requerimiento	Módulo	Descripción
RF-07	Gestión de proveedores	Proveedores	Registrar y administrar la información de los proveedores.
RF-08	Entradas y salidas	Inventario	Registrar entradas y salidas de inventario de un producto.
RF-09	Actualización de stock	Inventario	Actualizar el stock de forma consistente con cada movimiento.
RF-10	Trazabilidad	Movimientos	Registrar cada movimiento con usuario y fecha, y consultarlo.
RF-11	Generación de reportes	Reportes	Generar reportes de inventario y de movimientos.
RF-12	Alertas por bajo stock	Notificaciones	Emitir alertas automáticas cuando el stock cae bajo el mínimo.

Tabla 2. Requerimientos funcionales del proyecto. Elaboración propia (2026).

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

3. Descripción del proyecto web: requerimientos no funcionales

ID	Requerimiento	Atributo	Descripción
RNF-01	**Autenticación de usuarios**	Seguridad	Toda petición a la API, salvo el inicio de sesión y el health check, exige un token JWT válido.
RNF-02	**Autorización por roles**	Seguridad	Cada acción verifica que el rol (ADMINISTRADOR o MECÁNICO) tenga permiso; si no lo tiene, la petición se rechaza.
RNF-03	**Protección de credenciales**	Seguridad	Las contraseñas se guardan solo como hash con bcryptjs, nunca en texto plano.
RNF-04	**Consistencia de stock**	Integridad	Cada movimiento actualiza el stock y el historial en una sola transacción; el stock nunca queda negativo, incluso con registros simultáneos.
RNF-05	**Validación de datos de entrada**	Integridad	Los datos de entrada se validan con Zod antes de llegar a la lógica de negocio.
RNF-06	**Trazabilidad de movimientos**	Auditoría	Cada movimiento guarda usuario, fecha, stock anterior y resultante, y no puede editarse ni eliminarse.
RNF-07	**Modularización del backend**	Mantenibilidad	El backend se organiza en capas y en módulos por dominio, de modo que un cambio quede acotado a un módulo.

Tabla 3. Requerimientos no funcionales del proyecto. Elaboración propia (2026).

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

3. Descripción del proyecto web: requerimientos no funcionales

ID	Requerimiento	Atributo	Descripción
RNF-08	**Manejo centralizado de errores**	Mantenibilidad	Un middleware único captura los errores y responde con un formato uniforme.
RNF-09	**Tiempo de respuesta en consultas**	Rendimiento	La búsqueda de un producto por SKU o nombre responde en 2 s o menos.
RNF-10	**Disponibilidad del servicio**	Disponibilidad	La API ofrece un endpoint de health check para verificar que el servicio está activo.
RNF-11	**Tolerancia a fallos no controlados**	Fiabilidad	Un error no controlado no detiene el servidor ni deja datos a medias: la transacción se revierte.
RNF-12	**Escalabilidad de la arquitectura**	Escalabilidad	Notificaciones está desacoplado por eventos y puede separarse como servicio; la base puede migrar a PostgreSQL cambiando el proveedor de Prisma.
RNF-13	**Usabilidad de la interfaz**	Usabilidad	La interfaz web permite a los mecánicos registrar movimientos de inventario de forma simple y con la menor cantidad de pasos posible.
RNF-14	**Comunicación segura**	Seguridad	La comunicación entre el frontend y el backend se realiza sobre HTTPS, con control de origen mediante CORS.

Tabla 3. Requerimientos no funcionales del proyecto. Elaboración propia (2026).

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

4. Arquitectura del proyecto

Arquitectura en capas + monolito modular

- **Presentación:** SPA en React + Vite + TypeScript.
- **Negocio:** API REST en Node.js + Express + TypeScript.
- **Datos:** SQLite en desarrollo y PostgreSQL en producción, vía Prisma ORM.

Justificación

- Dependencia en un solo sentido: por eso interfaz y datos se construyeron en paralelo.
- Módulos por dominio: un cambio queda acotado a un módulo (RNF-07).
- Notificaciones desacoplado por eventos: puede separarse como servicio (RNF-12).

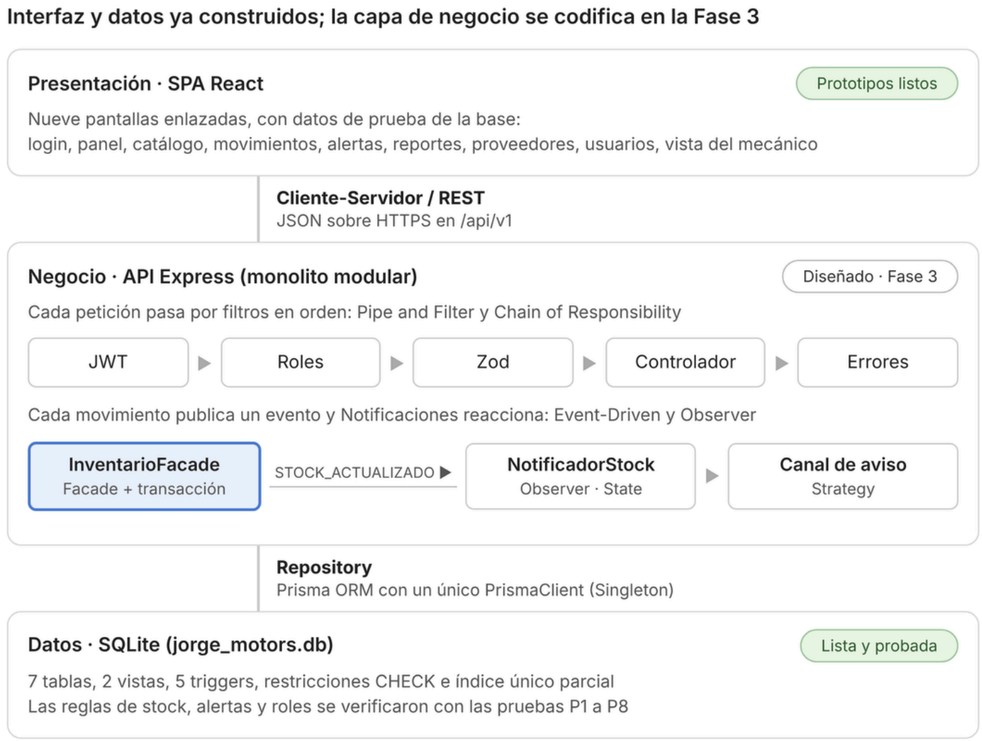


Figura 1. Arquitectura en capas y ubicación de cada patrón. Elaboración propia (2026).

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

4. Arquitectura del proyecto: capas y comunicación

Aplicación web (SPA)	API REST	Base de datos
Tecnología: React + Vite + TypeScript **Capa de presentación** Pantallas, formularios y navegación según el rol.	**Tecnología:** Node.js + Express + TypeScript **Capa de negocio** Reglas del taller, autenticación, transacciones de stock y envío de alertas.	**Tecnología:** SQLite / PostgreSQL vía Prisma ORM **Capa de datos** Persistencia de las 7 entidades e integridad del stock.

Flujo de una petición

Decisión clave: la API es el único contenedor que toca la base de datos; además envía los avisos al proveedor por correo, SMS o WhatsApp.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

4. Arquitectura del proyecto: patrones arquitectónicos

ID	Patrón	Capas	Aplicación en el proyecto
PA-01	Arquitectura en capas	Todas	Dependencia en un solo sentido: los prototipos no contienen reglas de negocio ni acceden a datos, y la base no conoce la interfaz.
PA-02	Cliente-Servidor / API REST	Presentación ↔ Negocio	Cada acción de la interfaz (confirmar una salida, enviar una alerta, guardar un proveedor) es una petición a /api/v1.
PA-03	Repository	Negocio ↔ Datos	schema.prisma mapea las 7 tablas a modelos; pasar de SQLite a PostgreSQL solo cambia el proveedor del esquema.
PA-04	Event-Driven	Negocio	El mecánico solo registra la salida: el evento de bajo stock genera la alerta en el panel y el tablero.
PA-05	Pipe and Filter	Negocio	Toda petición pasa por la cadena JWT → roles → Zod → controlador → manejo de errores.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

5. Patrones de diseño aplicados

Patrón	Tipo	Aplicación en el proyecto	Beneficio
Singleton	Creacional	Una sola instancia de PrismaClient (src/db/prisma.ts) reutilizada por todos los servicios.	Una única conexión a la base de datos; evita instancias duplicadas.
Facade	Estructural	InventarioFacade.registrarSalida() actualiza el stock y crea el movimiento en una transacción.	El usuario ve una sola acción, “Confirmar”; el stock nunca queda negativo (RNF-04).
Observer	Comportamiento	Inventario publica STOCK_ACTUALIZADO y Notificaciones se suscribe, sin conocerse entre sí.	Alertas automáticas sin acoplar módulos (RF-12, RNF-12).
Strategy	Comportamiento	CorreoStrategy, SmsStrategy y WhatsAppStrategy, elegidas según canal_preferido.	Canal de aviso intercambiable por proveedor.
State	Comportamiento	Clases Pendiente, Enviada y Atendida con las acciones enviar() y atender().	Solo transiciones válidas; un trigger en la base es la segunda barrera.
Chain of Responsibility	Comportamiento	Middlewares de Express en orden: JWT → roles → Zod → controlador → errores.	Seguridad y validación en filtros independientes.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

5. Patrones de diseño: diseño de referencia y avance

Facade + Observer + Strategy (TypeScript)

```
// inventario.facade.ts — Facade + transacción atómica (RNF-04)
export class InventarioFacade {
  async registrarSalida(dto: SalidaDTO, usuarioId: number) {
    const mov = await this.prisma.$transaction(async (tx) => {
      // descuento condicional: evita stock negativo
      const r = await tx.stock.updateMany({
        where: { productoId: dto.productoId,
                 cantidadActual: { gte: dto.cantidad } },
        data:  { cantidadActual: { decrement: dto.cantidad } },
      });
      if (r.count === 0) throw new StockInsuficienteError(dto.productoId);
      return tx.movimiento.create({ data: { /* stock anterior y resultante */ } });
    });
    // Observer: Inventario no conoce a Notificaciones
    this.eventos.emit("STOCK_ACTUALIZADO", { productoId: dto.productoId });
    return mov;
  }
}
// notificador-stock.ts — Observer + Strategy + State
eventos.on("STOCK_ACTUALIZADO", async ({ productoId }) => {
  const alerta = await alertaService.evaluarStockMinimo(productoId);
  if (alerta) await notificador.enviar(alerta); // canal del proveedor
});
```

Avance de los patrones

Aplicado en datos e interfaz

Facade, Observer, Strategy y State: tablas, CHECK, triggers e índices probados, y prototipos que los muestran.

Diseñado (código en Fase 3)

Singleton y Chain of Responsibility dependen del backend; ya tienen su diseño de referencia.

Trazabilidad

Cada patrón queda conectado al requerimiento que atiende (RF-12, RNF-04, RNF-07…) y a su evidencia en la base y en los prototipos.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

5. Patrones de diseño: matriz de trazabilidad

Patrón	Requerimiento	Evidencia en la base de datos	Evidencia en prototipos	Estado
Capas	RNF-07, RNF-12	Capa de datos completa: 7 tablas, 2 vistas	Prototipos completos	Datos e interfaz listos
Cliente-Servidor / REST	RNF-14	Solo la API accederá a jorge_motors.db	Acciones que consumirán /api/v1	Diseñado
Repository	RNF-07	schema.prisma validado, camelCase ↔ snake_case	—	Datos listos
Event-Driven	RF-12	alerta_stock + índice único parcial	Panel, movimientos, alertas	Datos e interfaz listos
Pipe and Filter	RNF-01, 02, 05	CHECK de rol	Inicio de sesión, usuarios	Diseñado
Singleton	RNF-09	Un único schema.prisma y una sola base	—	Diseñado
Facade	RNF-04	CHECK de stock ≥ 0 y aritmética	Registro de movimientos	Datos e interfaz listos
Observer	RF-12	alerta_stock, sin duplicados	Panel, alertas, vista mecánico	Datos e interfaz listos
Strategy	RF-12	canal_preferido, canal, CHECK por canal	Alertas, proveedores	Datos e interfaz listos
State	RF-12	estado + trigger trg_alerta_transicion	Tablero de alertas	Datos e interfaz listos
Chain of Responsibility	RF-02, 03, RNF-08	CHECK de rol	Inicio de sesión, usuarios	Diseñado

Tabla 8. Matriz de trazabilidad entre patrones, requerimientos y evidencias. Elaboración propia (2026).

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

6. Modelado arquitectónico: modelo C4

¿Por qué C4?

- **Niveles progresivos:** se explica el sistema al dueño del taller y al equipo de desarrollo.
- **Notación simple:** cajas, personas y flechas etiquetadas, sin la complejidad de UML.
- **Coherente con las capas:** los contenedores son SPA, API y base de datos.
- **Continuidad:** las 7 clases del nivel 4 son las 7 tablas, y cada componente tiene su pantalla.
- **Trazabilidad:** justifica visualmente decisiones como el desacople de Notificaciones.

Nivel 1 · Contexto

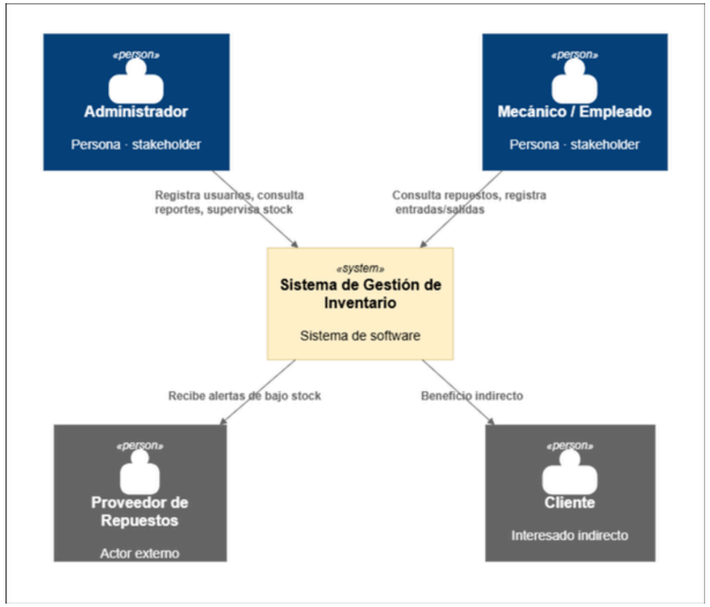
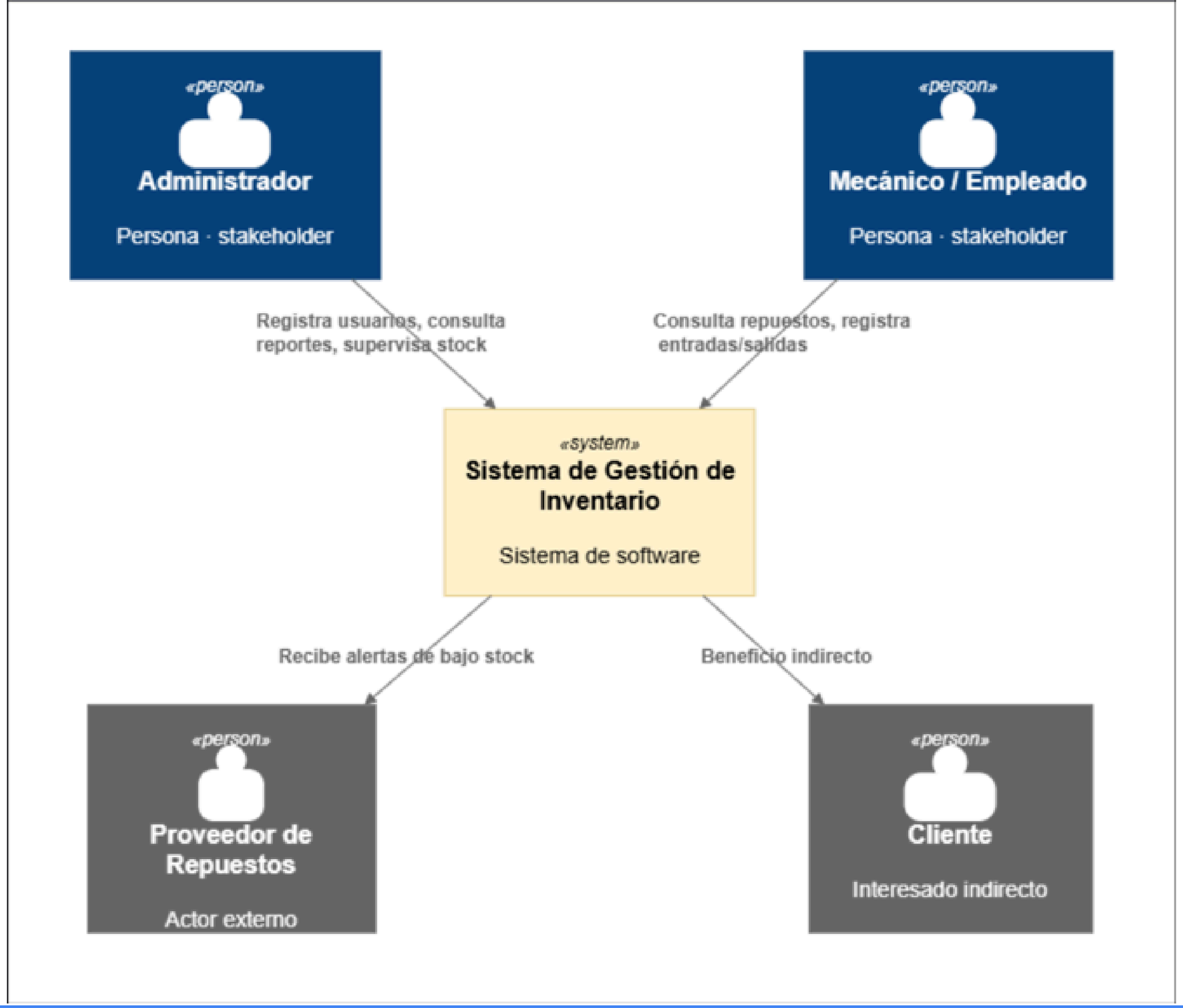


Figura 2. Vista de contexto (C4 – Nivel 1). Elaboración propia (2026).

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados



Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

6. Modelado arquitectónico: modelo C4

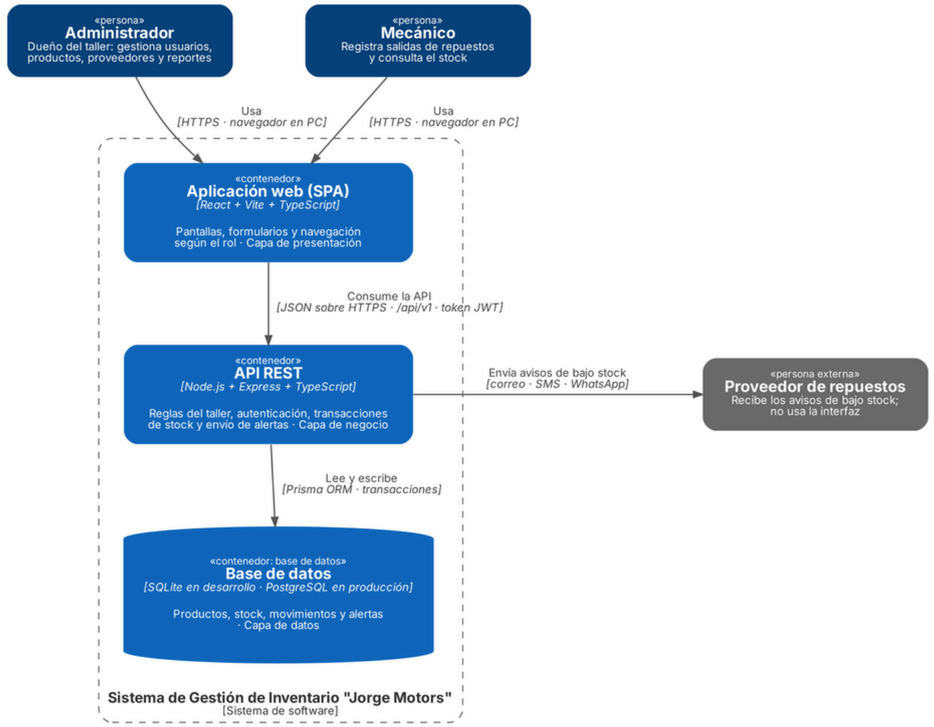


Figura 3. Vista de contenedores (C4 – Nivel 2). Elaboración propia (2026).

Nivel 2 · Contenedores

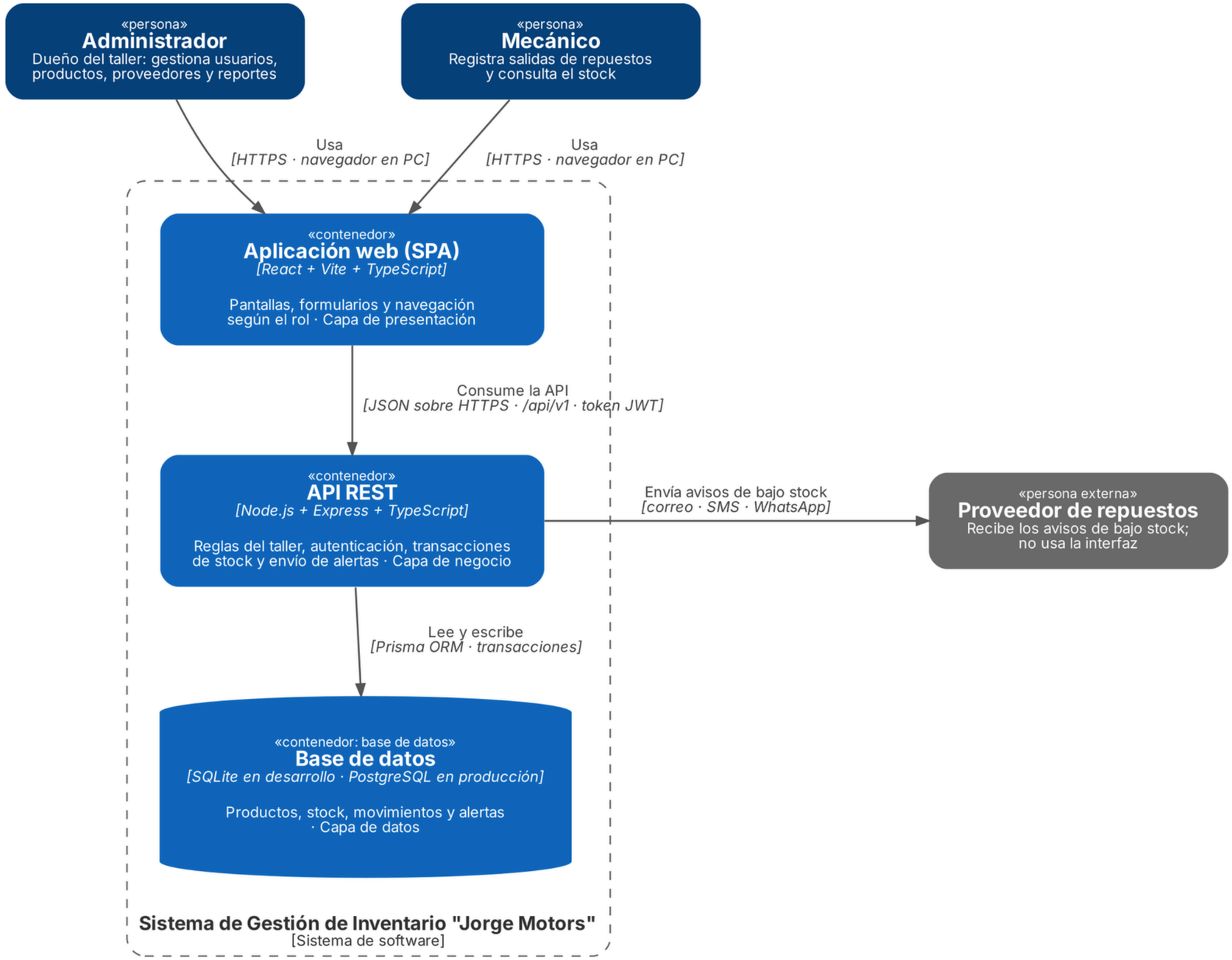
Propósito

Abre la caja del sistema y muestra las tres piezas que se despliegan por separado: aplicación web (SPA), API REST y base de datos. Cada una corresponde a una capa de la arquitectura.

Decisión que muestra

La API es el único contenedor que toca la base de datos. Así se centralizan la validación, la seguridad y el control de concurrencia del stock en un solo lugar.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados



Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

6. Modelado arquitectónico: modelo C4

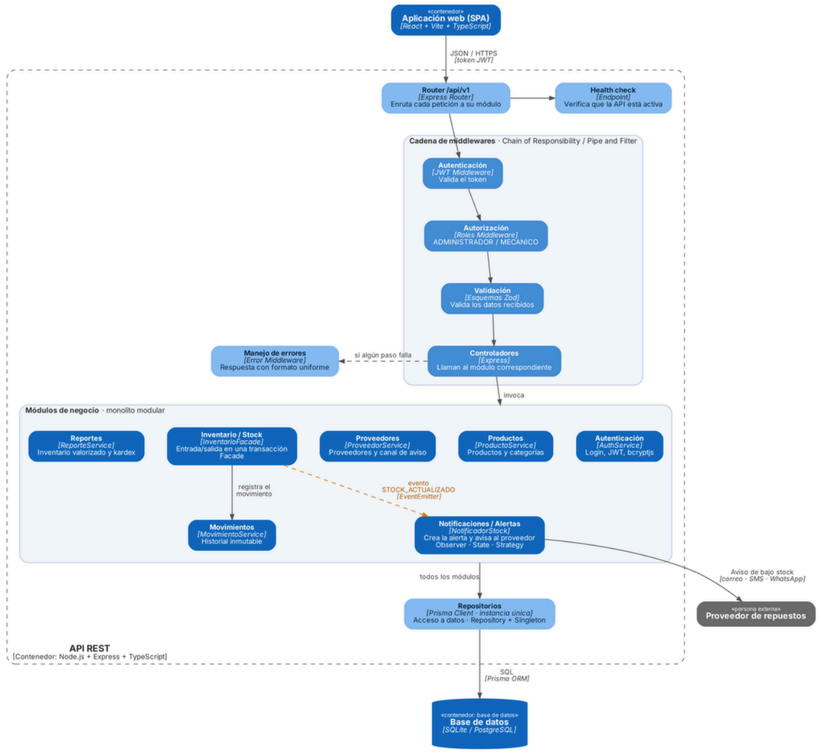


Figura 4. Componentes de la API REST (C4 – Nivel 3). Elaboración propia (2026).

Nivel 3 · Componentes de la API

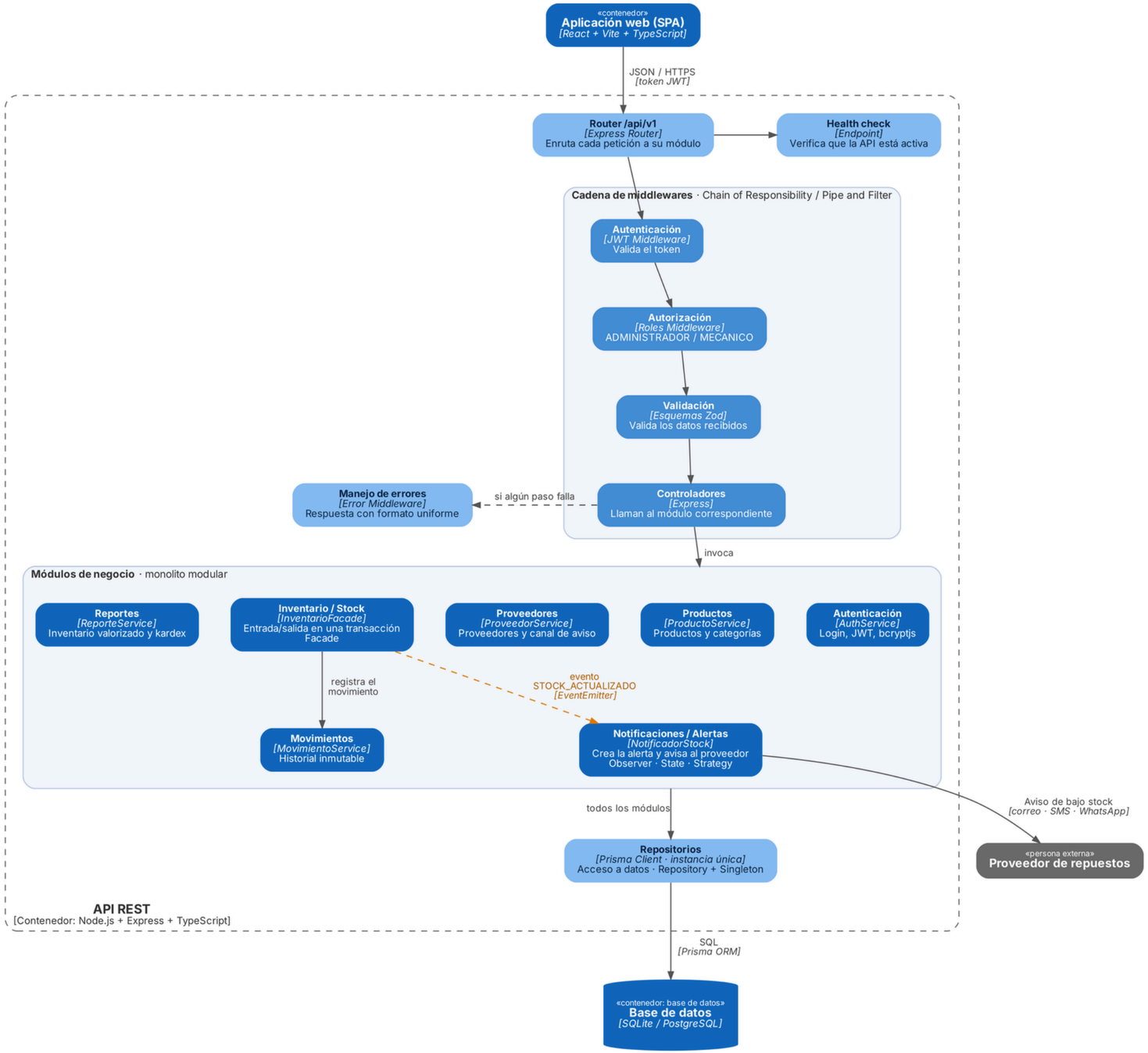
Cadena de middlewares (Chain of Responsibility)

JWT	Roles	Zod	Controlador	Errores
sesión iniciada (RNF-01)	permiso del rol (RNF-02)	datos válidos (RNF-05)	ejecuta el módulo	formato uniforme (RNF-08)

7 módulos de negocio: Autenticación/Usuario, Productos, Proveedores, Inventario/Stock, Movimientos, Reportes y Notificaciones/Alertas.

Decisión: Inventario solo publica STOCK_ACTUALIZADO y Notificaciones reacciona. Los avisos cambian sin tocar el registro de movimientos (RNF-12).

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados



Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

6. Modelado arquitectónico: modelo C4

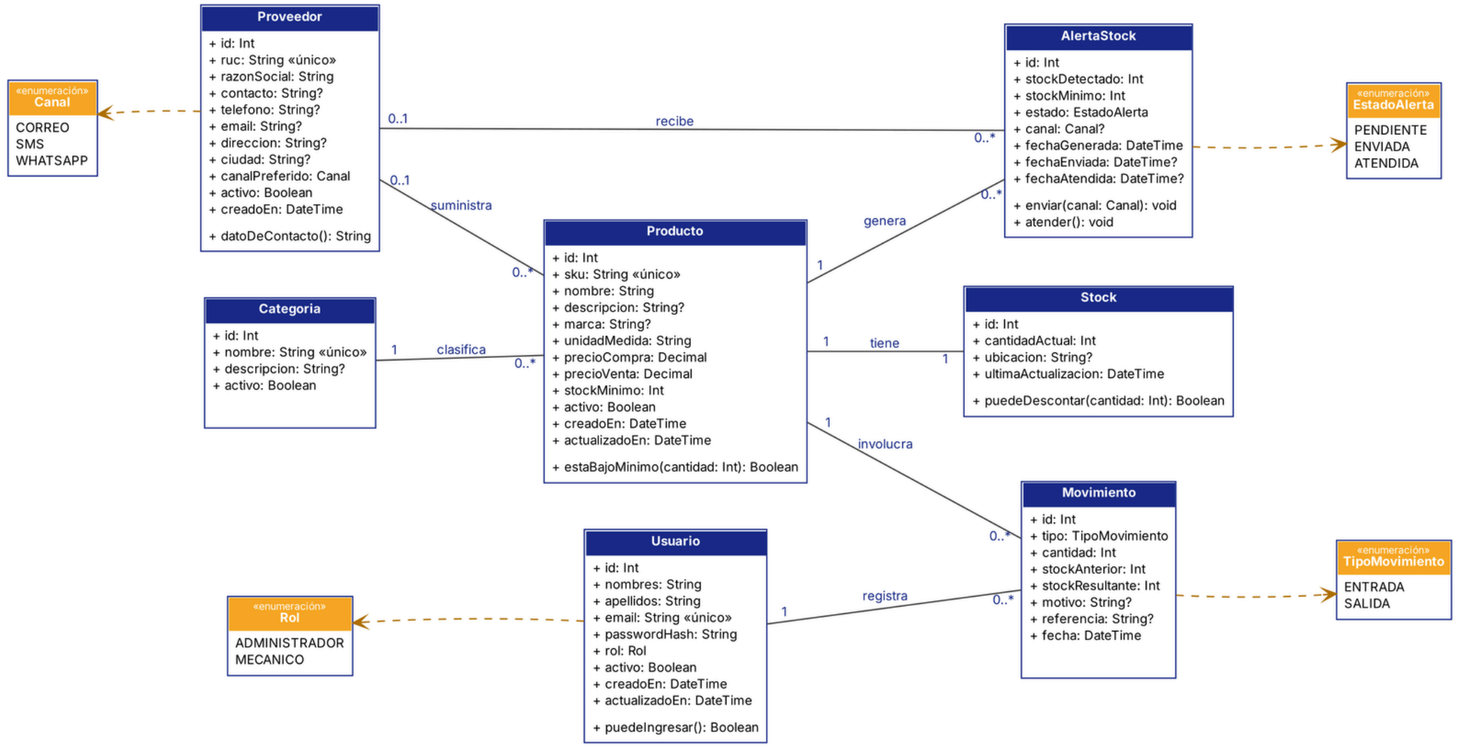


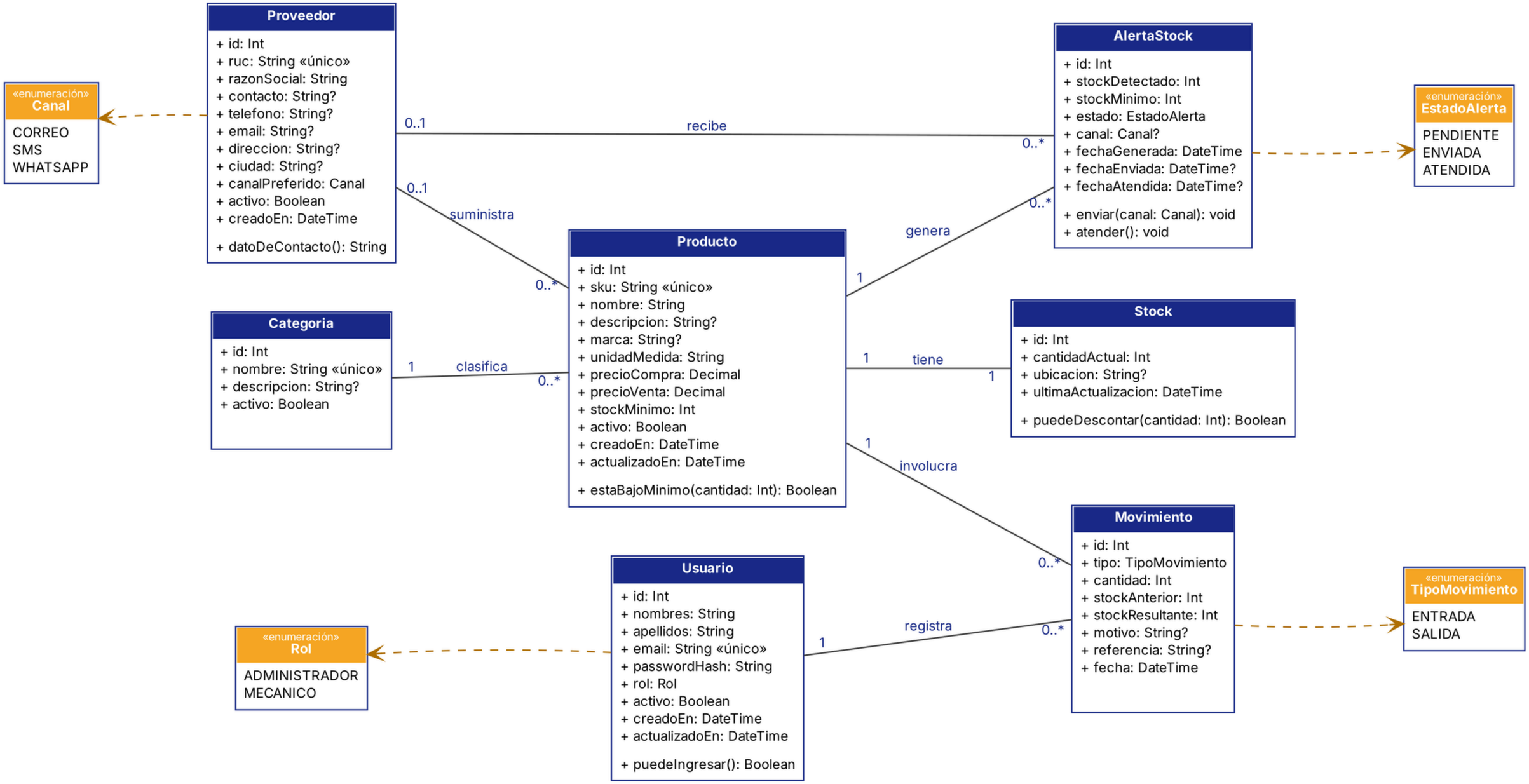
Figura 5. Diagrama de clases del dominio (C4 – Nivel 4). Elaboración propia (2026).

Nivel 4 · Código

7 clases = 7 tablas
Usuario, Categoria, Proveedor, Producto, Stock, Movimiento y AlertaStock.

Mismos nombres
camelCase en el código y snake_case en la base. Enums Rol, Canal, TipoMovimiento y EstadoAlerta.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados



Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

6. Modelado arquitectónico: modelo de datos

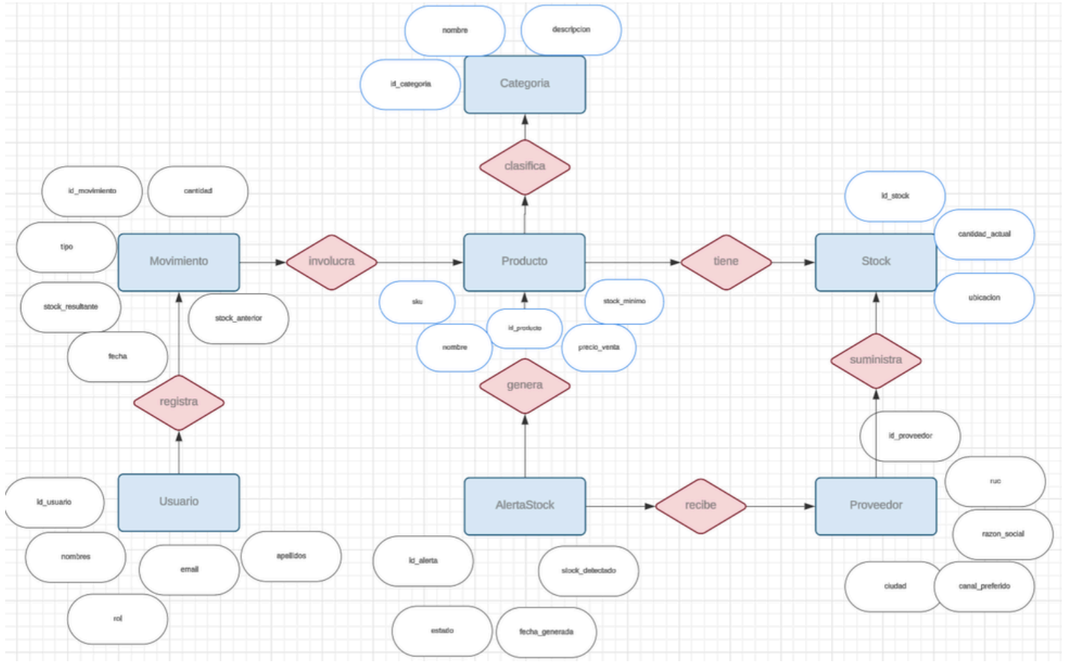


Figura 6. Modelo conceptual (notación de Chen).

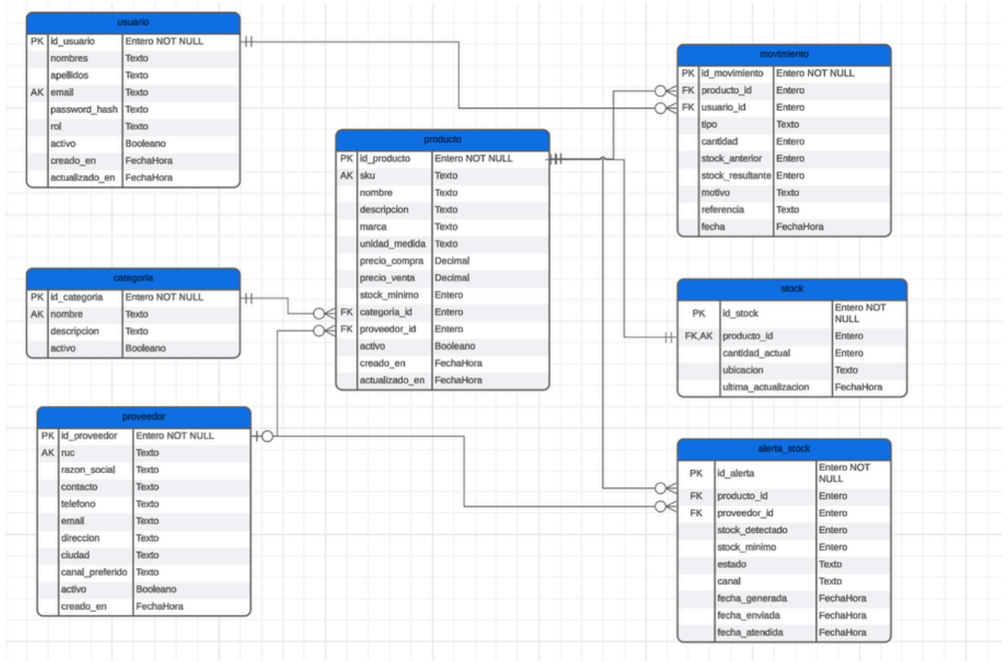


Figura 7. Modelo lógico relacional (PK, FK y cardinalidades).

Tercera forma normal: Redundancias solo intencionales, para un historial auditable (RNF-06).

SQLite 3: 7 tablas, 2 vistas (v_inventario, v_kardex), CHECK, triggers e índices.

8 pruebas de integridad: 1 operación válida ejecutada y 7 rechazadas por la propia base.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

6. Modelado arquitectónico: decisiones del modelo físico

Decisión física (SQLite 3)	Por qué se tomó	Respalda
Tabla stock separada de producto, CHECK (cantidad_actual ≥ 0)	El stock cambia en cada movimiento: se aísla esa escritura y la base impide un saldo negativo.	**RNF-04**
CHECK stock_resultante = stock_anterior ± cantidad	No se puede guardar un movimiento descuadrado, aunque el código falle.	**RNF-04, RNF-06**
Triggers de inmutabilidad en movimiento	El historial funciona como un kardex: no se edita ni se borra.	**RNF-06**
CHECK de rol (ADMINISTRADOR o MECANICO) y password_hash obligatorio	Solo existen dos perfiles válidos y la contraseña nunca queda en texto plano.	**RF-02, RNF-02, RNF-03**
canal_preferido en proveedor y canal en alerta_stock, CHECK por canal	Cada canal exige su dato: correo para CORREO, teléfono para SMS o WhatsApp.	**Strategy**
estado de la alerta + trigger trg_alerta_transicion	Solo se permite Pendiente → Enviada → Atendida.	**State**
Índice único parcial uq_alerta_abierta_por_producto	Un producto tiene como máximo una alerta abierta: no hay avisos duplicados.	**Observer**
sku único e índices en nombre, fecha y producto	La búsqueda de un repuesto no recorre toda la tabla.	**RNF-09 (≤ 2 s)**
Vistas v_inventario y v_kardex	Los reportes se arman desde las tablas, sin duplicar datos.	**RF-11**
Fechas en texto ISO 8601 (UTC)	Es el formato que usa Prisma; la interfaz muestra la hora de Lima.	**Compatibilidad**
PRAGMA foreign_keys = ON	SQLite no valida las claves foráneas si no se activa.	**Integridad**

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

6. Modelado arquitectónico: pruebas de integridad

Prueba	Qué se intenta	Mecanismo que responde	Resultado	Qué protege
P1	Salida de 2 unidades (stock 5 → 3) en una transacción	Transacción: stock y movimiento se guardan juntos	**Se ejecuta**	RNF-04 (Facade)
P2	Dejar el stock en negativo	CHECK (cantidad_actual ≥ 0)	**Rechazada**	RNF-04
P3	Guardar un movimiento con aritmética incorrecta	CHECK de stock_resultante	**Rechazada**	RNF-04, RNF-06
P4	Editar o borrar un movimiento	Triggers de inmutabilidad	**Rechazada**	RNF-06 (kardex)
P5	Crear una segunda alerta abierta para el mismo producto	Índice único parcial	**Rechazada**	Observer sin duplicados
P6	Pasar una alerta de PENDIENTE a ATENDIDA sin enviarla	Trigger trg_alerta_transicion	**Rechazada**	State
P7	Crear un usuario con un rol inexistente	CHECK de rol	**Rechazada**	RF-02, RNF-02
P8	Crear un producto con una categoría inexistente	Clave foránea (foreign_keys = ON)	**Rechazada**	Integridad referencial

8 de 8 pruebas con el resultado esperado: 1 ejecutada y 7 rechazadas.

Segunda barrera: aunque el backend tenga un error, la base no acepta datos inválidos.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

6. Modelado arquitectónico: consistencia del stock y concurrencia

registrarSalida(): qué ocurre en una salida de inventario (Facade + transacción)

1 **Inicia la transacción**
prisma.$transaction

→

2 **Descuento condicional**
solo si cantidadActual ≥ cantidad

→

3 **¿Se descontó?**
si no: StockInsuficienteError y rollback

→

4 **Crea el movimiento**
con stock anterior y resultante

→

5 **Confirma (commit)**
stock e historial juntos

→

6 **Publica el evento**
STOCK_ACTUALIZADO → Notificaciones

Ejemplo: dos mecánicos a la vez

Hay 3 filtros. El mecánico A pide 2 y el mecánico B pide 2 al mismo tiempo.
A: se descuenta (3 → 1) y se guarda su movimiento.
B: la condición ya no se cumple (1 < 2): no se descuenta nada y recibe “stock insuficiente”.

Por qué así

- El descuento y la verificación son una sola operación: no hay un momento en que dos usuarios lean el mismo stock y lo descuenten dos veces.
- La alerta se genera después del commit: si la salida falla, no se avisa al proveedor por error.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

6. Modelado arquitectónico: diccionario de datos

usuario

Personas que usan el sistema
email UNIQUE · password_hash (bcryptjs) · rol ADMINISTRADOR/MECANICO · activo

categoria

Clasificación de productos
nombre UNIQUE: Repuestos, Lubricantes, Filtros, Baterías, Herramientas…

proveedor

Empresas que reponen el stock
ruc UNIQUE (11 dígitos) · razon_social · canal_preferido CORREO/SMS/WHATSAPP

producto

Catálogo de repuestos e insumos
sku UNIQUE (FIL-0001) · unidad_medida · precios ≥ 0 · stock_minimo · FK categoría

stock

Existencias actuales (1:1)
producto_id UNIQUE · cantidad_actual ≥ 0 · ubicacion (B-01)

movimiento

Historial inmutable
tipo ENTRADA/SALIDA · cantidad > 0 · stock anterior y resultante · motivo · fecha

alerta_stock

Avisos de reabastecimiento
estado PENDIENTE → ENVIADA → ATENDIDA · canal · fechas generada/enviada/atendida

7 tablas · 2 vistas
Tercera forma normal; FK con RESTRICT, SET NULL o CASCADE según la regla del negocio.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

6. Modelado arquitectónico: claves foráneas y normalización

Tabla hija	Clave foránea	Tabla padre	Al borrar el padre	Motivo
producto	categoria_id	categoria	**RESTRICT**	No pueden quedar productos sin categoría.
producto	proveedor_id	proveedor	**SET NULL**	Un producto puede no tener proveedor (herramientas propias).
stock	producto_id (única)	producto	**CASCADE**	Existencias sin producto no tienen sentido.
movimiento	producto_id	producto	**RESTRICT**	El historial se conserva.
movimiento	usuario_id	usuario	**RESTRICT**	Siempre se sabe quién registró cada movimiento.
alerta_stock	producto_id	producto	**CASCADE**	La alerta depende del producto.
alerta_stock	proveedor_id	proveedor	**SET NULL**	La alerta queda sin destinatario, pero no se pierde.

Tercera forma normal: cada atributo depende solo de la clave de su tabla.

Redundancia intencional: stock anterior/resultante y stock mínimo se guardan "congelados" para auditar (RNF-06).

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: cobertura de requerimientos funcionales

RF	Requerimiento	Pantalla(s)	Cómo se evidencia
RF-01	Inicio de sesión	01 Inicio de sesión	Correo y contraseña; con credenciales válidas se obtiene el token JWT
RF-02	Diferenciación de roles	01, 08 y vista del mecánico	Cada rol entra a su propia vista y menú
RF-03	Restricción por rol	08 Usuarios y roles, 10 Consultar stock	Matriz de permisos; el mecánico no ve precios ni opciones de gestión
RF-04	Usuario autenticado	Todas (menú lateral)	Nombre y rol de quien inició sesión siempre visibles
RF-05	Gestión de productos	03 Productos y stock	Catálogo con búsqueda, alta y edición
RF-06	Gestión de categorías	03 Productos y stock	Filtro por categoría y botón Categorías
RF-07	Gestión de proveedores	07 Proveedores	Lista y formulario con canal de aviso
RF-08	Entradas y salidas	04 Registrar, 09 Mecánico	Formulario en 3 pasos con orden de trabajo
RF-09	Actualización de stock	03, 04, 10	Stock resultante (5 → 3) antes de confirmar; estado frente al mínimo
RF-10	Trazabilidad	02, 06 Kardex, 09 Mis últimas salidas	Cada movimiento con usuario, fecha y documento
RF-11	Reportes	02 Panel, 06 Reportes	Inventario valorizado y kardex exportables
RF-12	Alertas por bajo stock	02, 04, 05 Alertas	Alerta automática y tablero Pendiente → Enviada → Atendida

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: navegación por rol

Mapa de navegación por rol

Figura 9. El administrador recorre todos los módulos; el mecánico, un ciclo de 3 pasos.

¿Cómo cumple?

RF-02 · Diferenciación de roles
Un único inicio de sesión lleva a cada usuario a su vista: el administrador al panel con menú lateral completo; el mecánico a Registrar salida.

RF-03 · Restricción por rol
El mecánico solo tiene dos opciones: Registrar salida y Consultar stock. La API además rechaza cualquier acción que su rol no permita.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: inicio de sesión

01 · Inicio de sesión

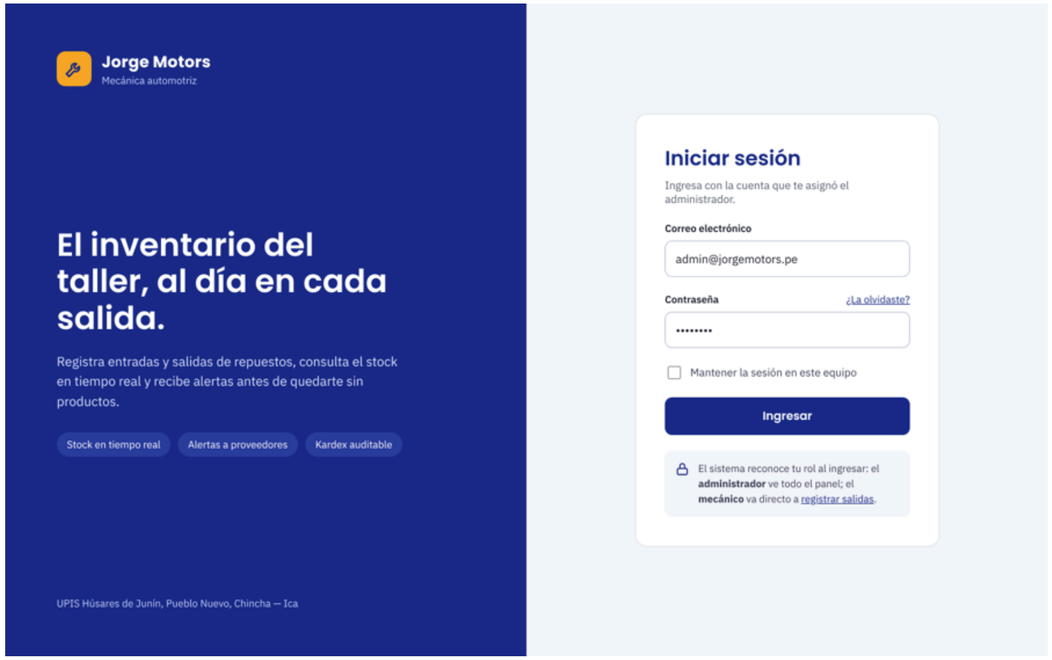


Figura 10. Primer eslabón de la cadena de seguridad.

¿Cómo cumple?

RF-01 · Inicio de sesión
Pide correo y contraseña. La API compara la contraseña con su hash (bcryptjs) y, si es válida, emite un token JWT.

RF-02 · Diferenciación de roles
Al ingresar, el sistema lee el rol del usuario y lo envía a su vista: panel o registro de salida.

RF-04 · Usuario autenticado
Con el token, la interfaz consulta quién inició sesión y muestra su nombre y rol en el menú lateral.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: panel del administrador

02 · Panel del administrador

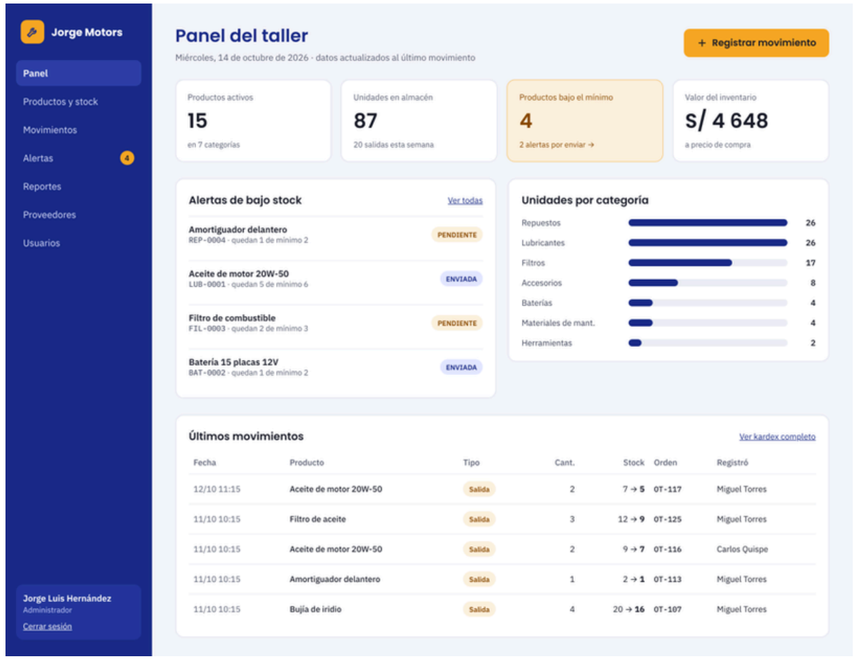


Figura 11. Resumen del estado del taller.

¿Cómo cumple?

RF-11 · Reportes
Cuatro indicadores: productos activos, unidades en almacén, productos bajo el mínimo y valor del inventario; más unidades por categoría.

RF-12 · Alertas por bajo stock
Muestra las alertas abiertas, generadas solas cuando un producto cae bajo su mínimo.

RF-10 · Trazabilidad
Lista los últimos movimientos con el usuario que los registró.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: productos y stock

03 · Productos y stock

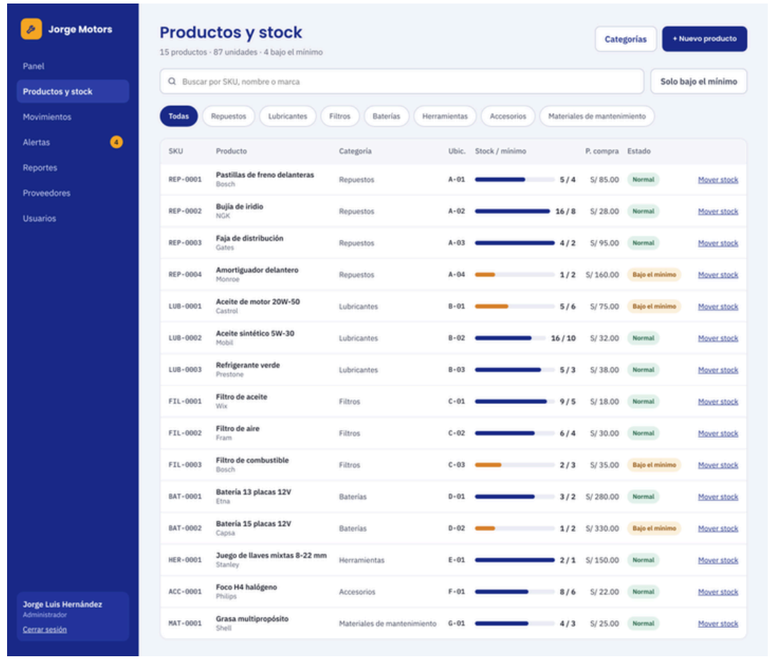


Figura 12. Catálogo con búsqueda, filtros y estado del stock.

¿Cómo cumple?

RF-05 · Gestión de productos
Catálogo con búsqueda por SKU o nombre y botón para registrar o editar productos.

RF-06 · Gestión de categorías
Filtro por categoría (Repuestos, Lubricantes, Filtros…) y acceso a la gestión de categorías.

RF-09 · Actualización de stock
Cada fila muestra el stock frente al mínimo, con barra y estado en texto: “Normal” o “Bajo el mínimo”.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: registrar entrada o salida

04 · Registrar entrada o salida (Facade)

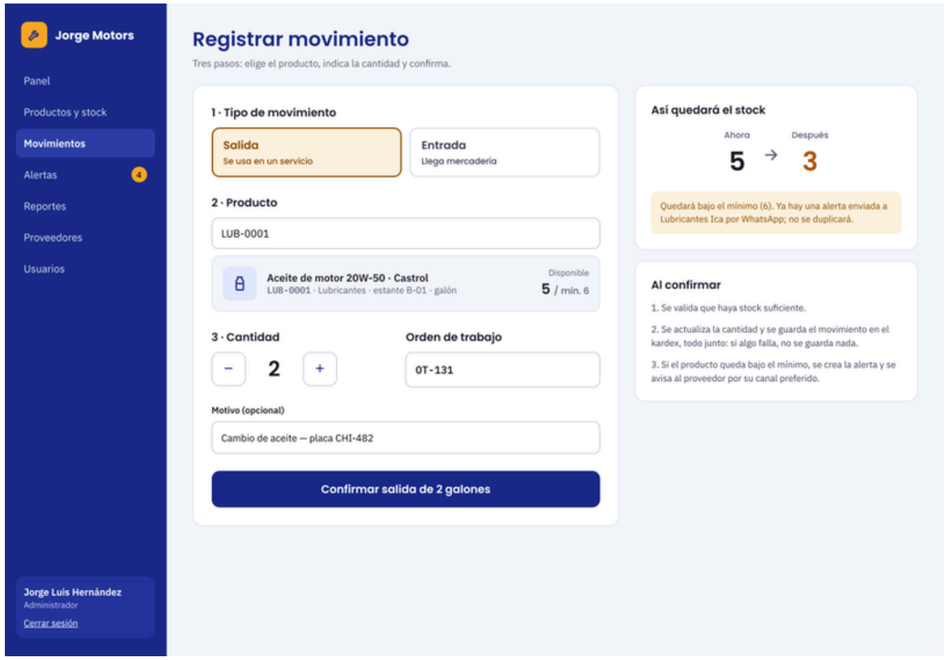


Figura 13. Formulario en tres pasos.

¿Cómo cumple?

RF-08 · Entradas y salidas
Tres pasos: tipo de movimiento, producto y cantidad, con orden de trabajo y motivo.

RF-09 · Actualización de stock
Antes de confirmar muestra el stock resultante (5 → 3). Al confirmar, stock y movimiento se guardan en una sola transacción.

RF-12 · Alertas por bajo stock
Si el stock quedará bajo el mínimo, avisa que el sistema notificará al proveedor.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: alertas de bajo stock

05 · Alertas de bajo stock (Observer, Strategy y State)

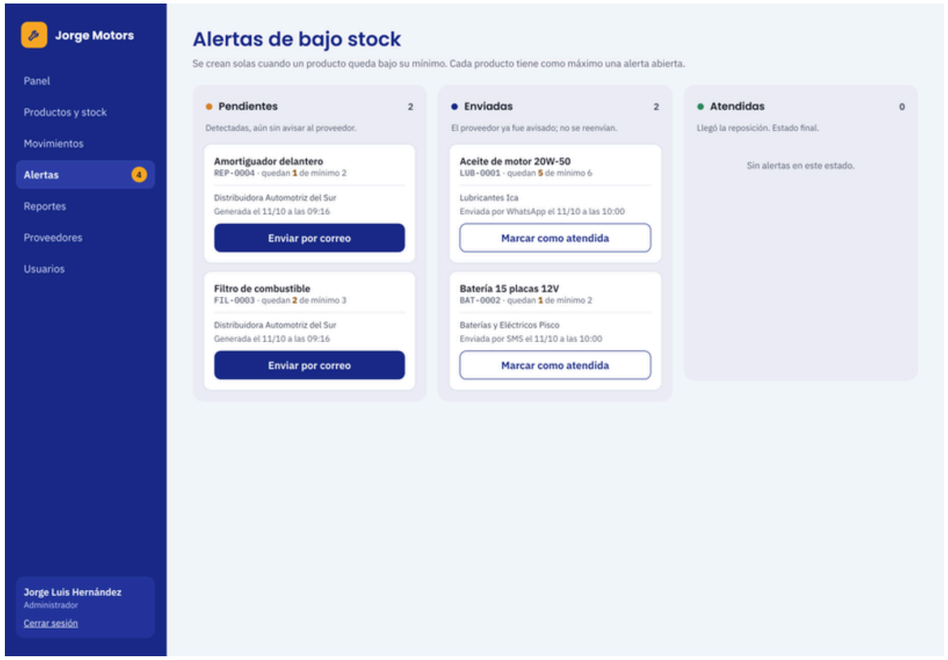


Figura 14. Tablero de tres columnas, una por estado.

¿Cómo cumple?

RF-12 · Generación automática

Al dejar el aceite 20W-50 en 3 unidades (mínimo 6) la alerta aparece sola, sin duplicarse (Observer).

RF-12 · Envío por el canal del proveedor

Una alerta pendiente se envía por correo, SMS o WhatsApp según el proveedor (Strategy).

RF-12 · Ciclo de vida

Pendiente → Enviada → Atendida: cada estado solo ofrece la acción que le corresponde (State).

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: reportes

06 · Reportes

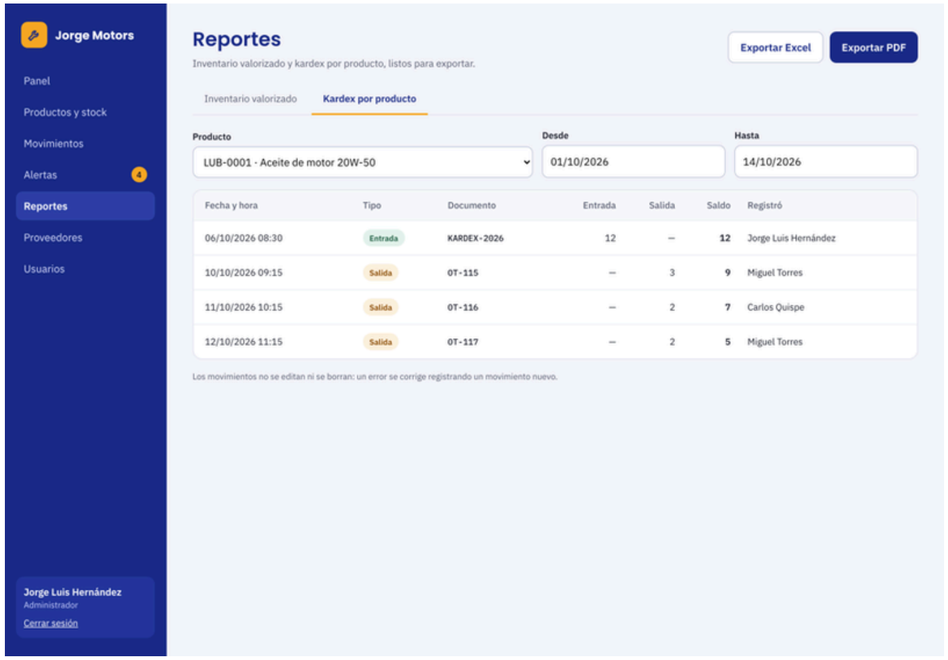


Figura 15. Kardex del aceite de motor 20W-50.

¿Cómo cumple?

RF-11 · Generación de reportes

Dos pestañas: inventario valorizado por categoría (S/ 4 648) y kardex por producto, exportables a Excel o PDF.

RF-10 · Trazabilidad

El kardex lista cada entrada y salida con su documento, el saldo resultante y quién la registró.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: proveedores

07 · Proveedores (Strategy)

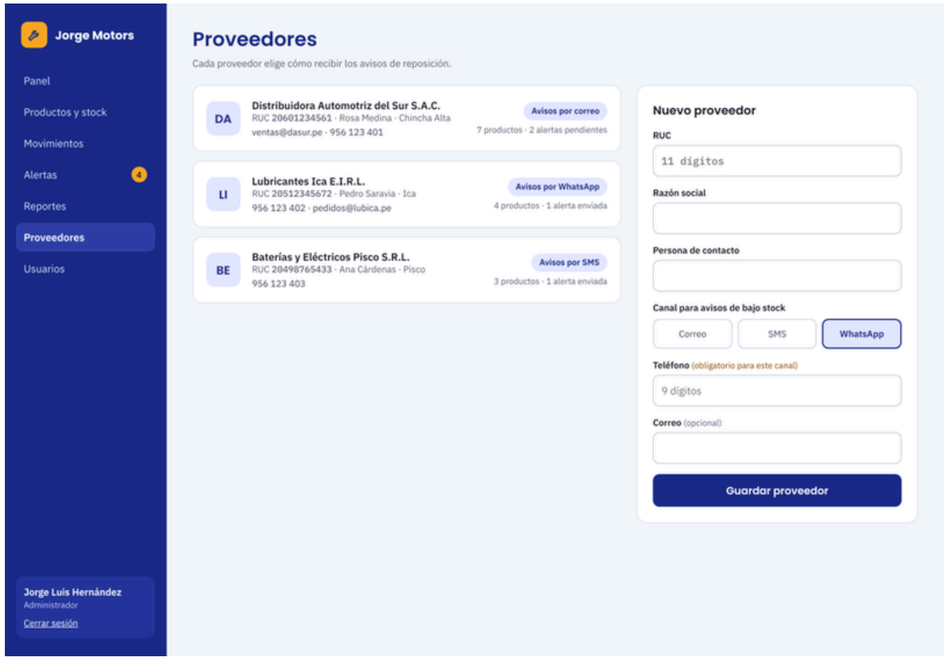


Figura 16. Lista y formulario de proveedores.

¿Cómo cumple?

RF-07 · Gestión de proveedores
Lista con RUC, razón social y canal de aviso, y formulario para registrar o editar.

RF-07 · Dato según el canal
Al elegir correo, SMS o WhatsApp, el formulario marca como obligatorio el dato que ese canal necesita, igual que la regla CHECK de la base.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: usuarios y roles

08 · Usuarios y roles (Chain of Responsibility)

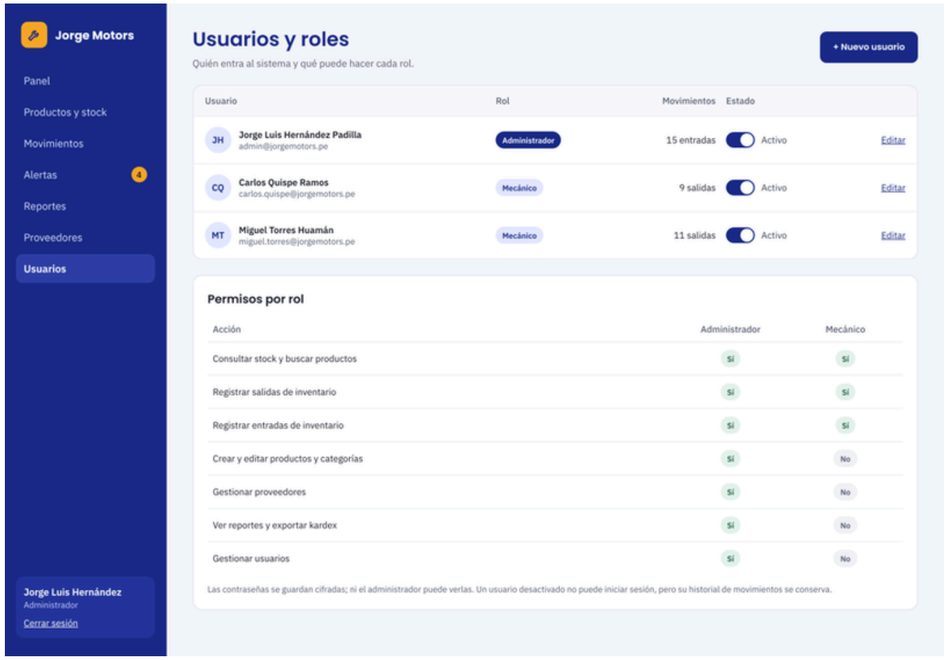


Figura 17. Usuarios y matriz de permisos.

¿Cómo cumple?

RF-02 · Diferenciación de roles
Cada usuario tiene rol ADMINISTRADOR o MECÁNICO, su número de movimientos y estado activo o inactivo.

RF-03 · Restricción por rol
La matriz de permisos define qué puede hacer cada rol; es la que aplicará el filtro de roles de la API.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: vista del mecánico

09 · Registrar salida: vista del mecánico

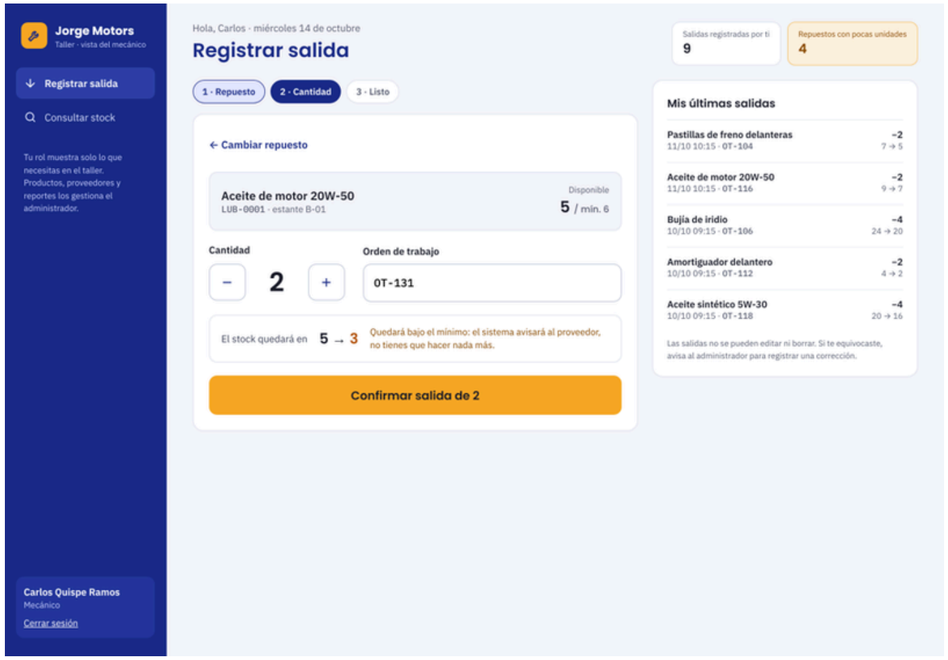


Figura 19. Paso 2: cantidad, orden de trabajo, stock resultante e historial propio.

¿Cómo cumple?

RF-08 · Registro de salidas
Tres pasos con indicador de progreso: elegir el repuesto, indicar cantidad y orden de trabajo, y confirmar.

RF-10 · Trazabilidad
El panel “Mis últimas salidas” muestra fecha, orden de trabajo y stock antes y después de lo que él registró.

RF-12 · Alertas
Si la salida deja el repuesto bajo el mínimo, se le informa que el sistema avisará al proveedor.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: consulta de stock del mecánico

10 · Consultar stock

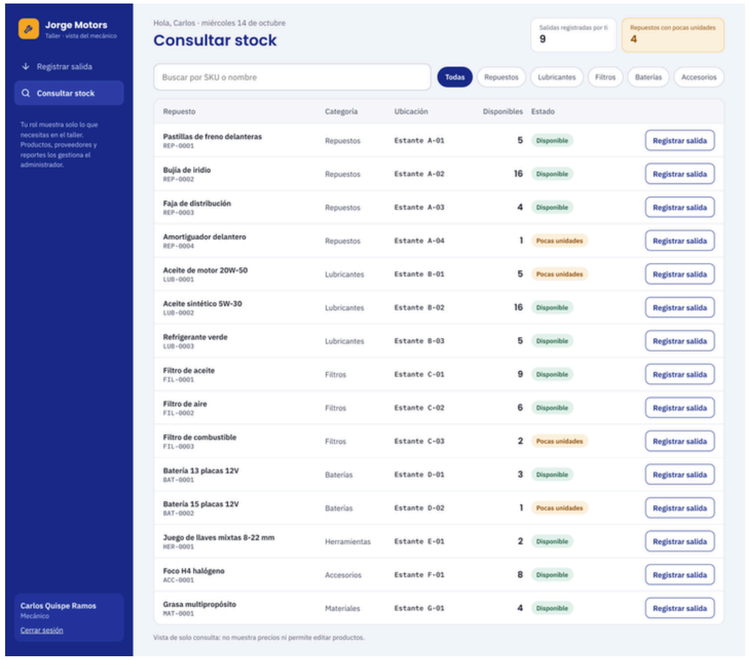


Figura 20. Consulta de stock por categoría y ubicación.

¿Cómo cumple?

RF-09 · Stock actualizado
Muestra categoría, estante, unidades disponibles y estado ("Disponible" o "Pocas unidades").

RF-03 · Restricción por rol
Es de solo lectura: no muestra precios ni permite editar, porque el rol mecánico no gestiona productos.

RF-08 · Acceso directo
Cada fila tiene "Registrar salida", que lleva al paso 2 con el repuesto ya elegido: solo dos acciones.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

7. Demostración del sistema: criterios de diseño de la interfaz

1. Navegación según el rol
Un único inicio de sesión; el mecánico solo ve las dos opciones que necesita (RF-02).

2. Menos pasos
Una salida toma 3 pasos y muestra el stock resultante antes de confirmar (RNF-13).

3. Estados claros
“Bajo el mínimo”, “Pendiente”: el estado se indica con texto, no solo con color.

4. Identidad visual
Azul #1A2B88 y naranja #F7A823; Poppins para títulos e IBM Plex Sans para el texto.

5. Accesibilidad básica
Botones y campos de al menos 48 px, contraste suficiente y etiquetas en todos los campos.

Prototipos navegables de alta fidelidad: 9 pantallas para PC (1440 px), con datos de prueba coherentes con la base: 15 productos, 4 alertas y el kardex.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

8. Resultados

Arquitectura documentada	Datos e interfaz listos	Patrones justificados	Componentes clave (Fase 3)	Metas de calidad
Cuatro niveles C4, coherentes con RF-01 a RF-12 y RNF-01 a RNF-14.	Modelos conceptual, lógico y físico en SQLite y prototipos de las 9 pantallas.	Seis patrones GoF y cinco patrones arquitectónicos con su evidencia y trazabilidad.	JWT y roles, salidas con transacción atómica, trazabilidad y alertas al proveedor (15/10 – 11/11).	100 % de accesos no autorizados denegados, búsqueda en ≤ 2 s y 0 inconsistencias de stock.

Validación prevista (12/11 – 25/11): pruebas funcionales con métricas antes/después frente a Excel: tiempo de registro, tiempo de búsqueda y número de errores.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

9. Conclusiones

Conclusión 1. Del problema a requisitos
Tres problemas del control en Excel se tradujeron en 12 requerimientos funcionales y 14 no funcionales.

Conclusión 2. Arquitectura clara y flexible
Capas, monolito modular y C4 separan presentación, negocio y datos; un cambio queda acotado a un módulo.

Conclusión 3. Patrones con propósito
Singleton, Facade, Observer, Strategy, State y Chain of Responsibility responden a necesidades concretas.

Conclusión 4. Solución factible
Antecedentes con pila Node.js/TypeScript respaldan la propuesta y aportan métricas para validarla.

Conclusión 5. Diseño coherente
Base de datos y 9 prototipos terminados y alineados con C4; el backend y el frontend se desarrollan en la Fase 3.

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

10. Bibliografía y herramientas

Referencias

Adriazola Gallegos, A. I., y Kjuro Quispe, E. F. (2024). Sistema web para el control de inventario del área de almacén de la empresa VALNET PERÚ E.I.R.L., Ilo, 2023 [Tesis de pregrado, Universidad César Vallejo].
Bass, L., Clements, P., y Kazman, R. (2021). Software architecture in practice (4.ª ed.). Addison-Wesley Professional.
Brown, S. (2018). The C4 model for software architecture. https://c4model.com
Cevallos Suárez, A. A. (2026). Implementar un sistema web de gestión de inventarios … Transfismor [Tesis de grado, Universidad Politécnica Salesiana].
Galarza Arias, J. C. (2021). Implementación de una aplicación web para el taller Carvy Soluciones Automotrices. Módulo: Gestión de inventario [Examen complexivo, UPSE].
Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1995). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
Kruchten, P. B. (1995). The 4+1 view model of architecture. IEEE Software, 12(6), 42–50.
Malquichagua Chu, S. R. (2022). Diseño de un sistema de gestión de inventarios para un taller mecánico [Trabajo de suficiencia profesional, Universidad del Pacífico].
Martínez Canelo, M. (2024). Arquitectura de software: Qué es y fundamentos clave. Profile Software Services.

Herramientas

- React + Vite
- TypeScript
- Node.js + Express
- Prisma ORM
- SQLite 3 / PostgreSQL
- JWT + bcryptjs
- Zod
- draw.io
- DBeaver

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados

10. Anexos

Cronograma del proyecto	
Completado	**Avance 2 · Diseño** Requerimientos, arquitectura, patrones, C4, base de datos y prototipos.
15/10 – 11/11	**Fase 3 · Desarrollo** Backend y frontend, integración de componentes.
12/11 – 25/11	**Pruebas funcionales** Métricas antes/después frente a Excel.

Evidencia del repositorio

Insertar aquí la captura del repositorio
(historial de commits por integrante y Diagrama de Gantt)

Universidad Nacional de Cañete - UNDC 2026. Todos los derechos reservados